Unit 6 | Chapter 10: The Remodeler (Advanced JS & Game Logic)

Learning Targets:

- Technical: I can use the HTML5 Canvas API to render shapes, text, and image sprites dynamically. (NV 5.3.4)
- Technical: I can develop a functional game loop using requestAnimationFrame and state management to simulate reality. (NV 7.1.3)
- Technical: I can implement Object-Oriented Programming (OOP) using ES6 Classes to manage multiple interactive game objects. (NV 7.1.3)
- Technical: I can apply physics and math concepts including gravity, friction, and Axis-Aligned Bounding Box (AABB) collision detection. (NV 5.3.2)
- Employability: I can demonstrate analytical reasoning and cognitive flexibility to independently debug complex loops, mathematical logic, and performance lag. (WRS 1.2.3)
- Employability: I can demonstrate diligence, probity, and professional decorum by writing modular, well-documented code that uses standardized naming conventions for complex applications. (WRS 1.1.1, 1.1.2, 1.1.4)

Compelling Question:

How does a developer use mathematical logic and persistent loops to "remodel" the browser into a simulated world?

Score	Performance Indicators
4.0	**I can:** ☐ Optimize game performance for high-refresh-rate monitors by implementing Delta Time to normalize speed, demonstrating advanced analytical reasoning and systematic resolution. (NV 7.1.3, WRS 1.2.3) ☐ Design a Particle System or complex Parallax effect that manages dozens of object instances without performance lag using advanced OOP. (NV 5.3.4) ☐ Solve complex geometry problems, such as using the Pythagorean Theorem to calculate center-point distance for circular collision detection, demonstrating profound cognitive flexibility. (WRS 1.2.3) ☐ Maintain strict probity and professional decorum by writing enterprise-level class structures that are reusable and modular. (WRS 1.1.2, 1.1.4)
3.5	In addition to score 3.0 performance, partial success at score 4.0 content.
3.0	**I can:** ☐ Construct an ES6 Class with a constructor() and custom methods to encapsulate game object logic, demonstrating professional

	decorum in code architecture. (NV 7.1.3, WRS 1.1.4) ☐ Build a complete Game Loop (Clear ⟶ Update ⟶ Draw) that correctly uses requestAnimationFrame() to animate the canvas. (NV 7.1.3) ☐ Implement AABB Collision Detection by applying analytical reasoning to check the overlap of x, y, width, and height coordinates. (NV 5.3.2, WRS 1.2.3) ☐ Apply physics logic like Gravity (vertical velocity) and Bouncing (velocity inversion). ☐ Demonstrate intrinsic motivation and diligence by consistently refining logic without teacher intervention. (WRS 1.1.1)
2.5	No major errors or omissions regarding score 2.0 content, and partial success at score 3.0 content.
2.0	**I can** ☐ Define common terminology and identify syntax: ☐ requestAnimationFrame, getContext('2d'), clearRect, AABB, Sprite Sheet, Hitbox, Game State, this keyword, class, constructor, fillStyle, and beginPath. (NV 5.3.4, 7.1.3) ☐ Identify the importance of punctuality and diligence in complex projects (WRS 1.1.1), probity in writing original logic (WRS 1.1.2), professional decorum in naming (WRS 1.1.4), and applying analytical reasoning to mathematical code (WRS 1.2.3).
1.5	Partial success at score 2.0 content, and major errors or omissions at 3.0.
1.0	With help, partial success at score 2.0 and score 3.0 content.
0.5	With help, partial success at score 2.0 but not at score 3.0 content.
0.0	Even with help, no success or attempt made.